

Project 2: Regression and Symbolic Regression

Seth Cohen, Mahimna Patel, Krish Ranawat

October 6, 2026

Overview

This report extends the Project 1 exploratory analysis of three UCI datasets (Auto MPG, Concrete Compressive Strength, Airfoil Self-Noise) with regression and symbolic regression. Five techniques are evaluated using **ScalaTion**, Python's **Statsmodels**, and **PySR**, following the model allocation in the assignment:

1. **Regression** (ScalaTion + Statsmodels): all 3 datasets.
2. **Regularized Regression**, Ridge and Lasso (ScalaTion + Statsmodels): Auto MPG only.
3. **Forward Feature Selection** (ScalaTion): Auto MPG and Concrete.
4. **Transformed Regression**, log/sqrt/Box-Cox (ScalaTion): Auto MPG and Airfoil.
5. **Symbolic Regression** (ScalaTion + PySR): all 3 datasets.

Unless noted otherwise, models use every available predictor for a dataset (no manual feature pre-selection), so results here are not directly comparable to Project 1's two-feature simple regressions. Full console logs for every ScalaTion run are in `results/<dataset>/scalation_output.txt`; a compact digest of coefficients and QoF is in `results/<dataset>/scalation_digest.txt`.

Reproducing the results. The Python scripts in `python/` read `data/` relative to the repository and need `statsmodels` (and `pysr` for Section 5). To run the ScalaTion code, copy `scala/Project2.scala` into a ScalaTion app at `src/main/scala/project2/`, set the environment variable `PROJECT2_DATA_DIR` to this repository's `data/` folder, and run `sbt "runMain project2.project2_autoMPG"` (likewise `project2_concrete` and `project2_airfoil`).

1 Regression

1.1 Auto MPG

ScalaTion's `Regression` and Statsmodels' OLS were fit on `mpg` against all 7 predictors (`cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model_year`, `origin`). Both tools solve the same least-squares problem and, as expected, agree to 6 significant figures on every coefficient, with identical R^2 . This holds for all three datasets.

Table 1: Auto MPG multiple regression. The ScalaTion and Statsmodels coefficients agree to 6 significant figures. Standard errors and p -values are from Statsmodels.

Term	ScalaTion	Statsmodels	Std. err.	p -value
intercept	-17.2184	-17.2184	4.644	< 0.001
cylinders	-0.49338	-0.49338	0.3233	0.128
displacement	0.019896	0.019896	0.00752	0.008
horsepower	-0.016951	-0.016951	0.01379	0.220
weight	-0.0064740	-0.0064740	0.000652	< 0.001
acceleration	0.080576	0.080576	0.09884	0.415
model_year	0.75077	0.75077	0.05097	< 0.001
origin	1.4261	1.4261	0.2781	< 0.001

QoF: $R^2 = 0.8215$, adj. $R^2 = 0.8182$, RMSE = 3.294, $F(7,384) = 252.4$, $p < 0.001$. 5-fold cross-validation gives a very similar mean $R^2 = 0.809$ (fold range 0.79–0.84; see `scalation_output.txt`), so the model generalizes well and is not overfit.

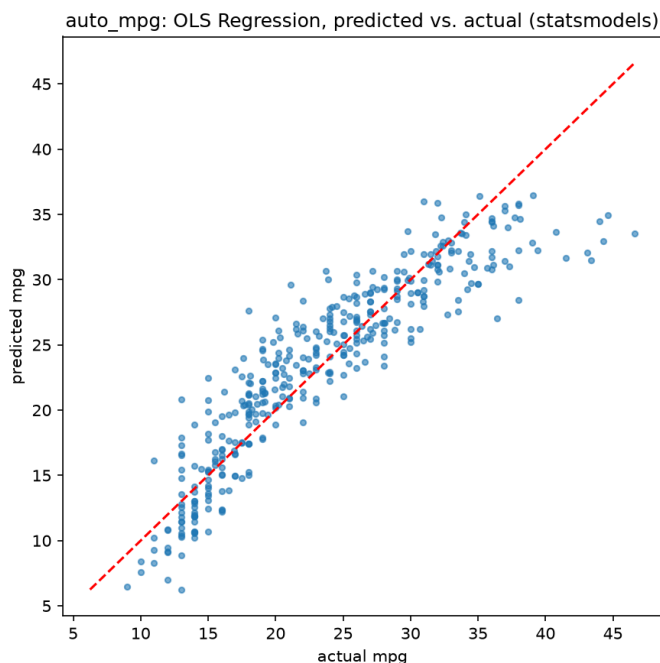


Figure 1: Auto MPG: Statsmodels OLS, predicted vs. actual mpg.

Interpretation: weight, model_year, and origin dominate: every extra 1000 lbs costs about

6.5 mpg, each model year (1970–1982) is worth about 0.75 mpg of efficiency gain, and, relative to US-built cars (origin 1), European cars (origin 2) are predicted about 1.4 mpg higher and Japanese cars (origin 3) about 2.9 mpg higher after controlling for size and era.¹ Displacement is the one predictor with a counter-intuitive sign: holding the other features fixed, its coefficient is positive (+0.0199 mpg per cubic inch) and significant ($p = 0.008$), even though on its own displacement is strongly negatively correlated with mpg. Cylinders’ coefficient has the expected negative sign but is not significant ($p = 0.13$). Both are symptoms of multicollinearity among the engine-size features (cylinders, displacement, horsepower and weight are all highly correlated with each other), so their individual coefficients are not reliable on their own.

1.2 Concrete Compressive Strength

Regression of `concrete_compressive_strength` on all 8 ingredient predictors.

Table 2: Concrete multiple regression. The ScalaTion and Statsmodels coefficients agree to 6 significant figures. Standard errors and p -values are from Statsmodels.

Term	ScalaTion	Statsmodels	Std. err.	p -value
intercept	−23.164	−23.164	26.59	0.384
cement	0.11979	0.11979	0.00849	< 0.001
blast_furnace_slag	0.10385	0.10385	0.01014	< 0.001
fly_ash	0.087943	0.087943	0.01259	< 0.001
water	−0.15030	−0.15030	0.04018	< 0.001
superplasticizer	0.29069	0.29069	0.09346	0.002
coarse_aggregate	0.018030	0.018030	0.00939	0.055
fine_aggregate	0.020155	0.020155	0.01070	0.060
age	0.11423	0.11423	0.00543	< 0.001

QoF: $R^2 = 0.6155$, adj. $R^2 = 0.6125$, RMSE = 10.35.

¹`origin` is categorical (1 = US, 2 = Europe, 3 = Japan) but, as in Project 1, it is entered as a single numeric column, which forces the origin 3 effect to be exactly twice the origin 2 effect. Dummy variables would remove that restriction.

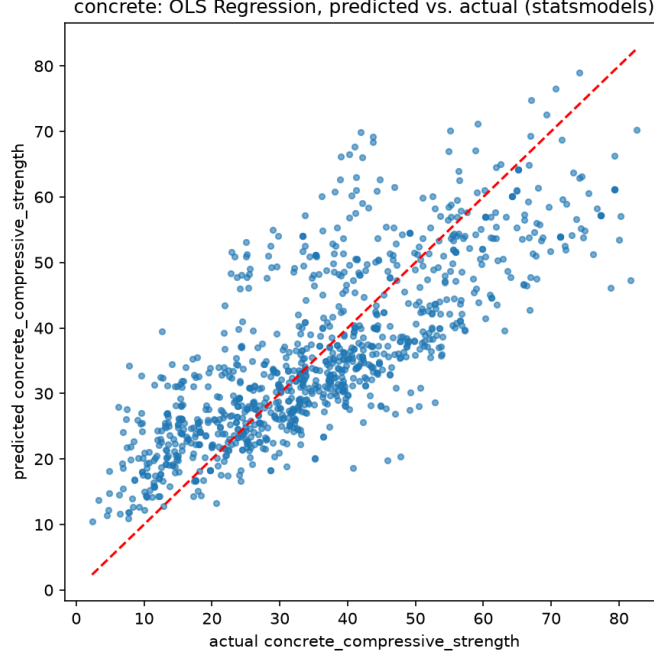


Figure 2: Concrete: Statsmodels OLS, predicted vs. actual compressive strength.

Interpretation: all three cementitious materials (cement, slag, fly ash) and superplasticizer raise strength, and water lowers it, which is the standard water/cement-ratio effect in concrete. Age has a positive effect because concrete keeps curing. The two aggregate fractions matter least: their coefficients are small and not significant at the 5% level ($p = 0.055$ and 0.060). With all 8 predictors the fit is about 2.5 times Project 1’s best single-feature fit ($R^2 \approx 0.25$ for cement alone), so strength depends on several ingredients at once. Still, 38% of the variance is unexplained, likely because of nonlinear curing and age interactions, which Section 5 takes up.

1.3 Airfoil Self-Noise

Regression of `scaled_sound_pressure_level` on all 5 predictors.

Table 3: Airfoil multiple regression. The ScalaTion and Statsmodels coefficients agree to 6 significant figures. Standard errors and p -values are from Statsmodels.

Term	ScalaTion	Statsmodels	Std. err.	p -value
intercept	132.834	132.834	0.5447	< 0.001
frequency	−0.0012822	−0.0012822	0.0000421	< 0.001
angle_of_attack	−0.42191	−0.42191	0.0389	< 0.001
chord_length	−35.688	−35.688	1.630	< 0.001
free_stream_velocity	0.099854	0.099854	0.00813	< 0.001
suction_side_displacement_thickness	−147.30	−147.30	15.01	< 0.001

QoF: $R^2 = 0.5157$, adj. $R^2 = 0.5141$, RMSE = 4.799.

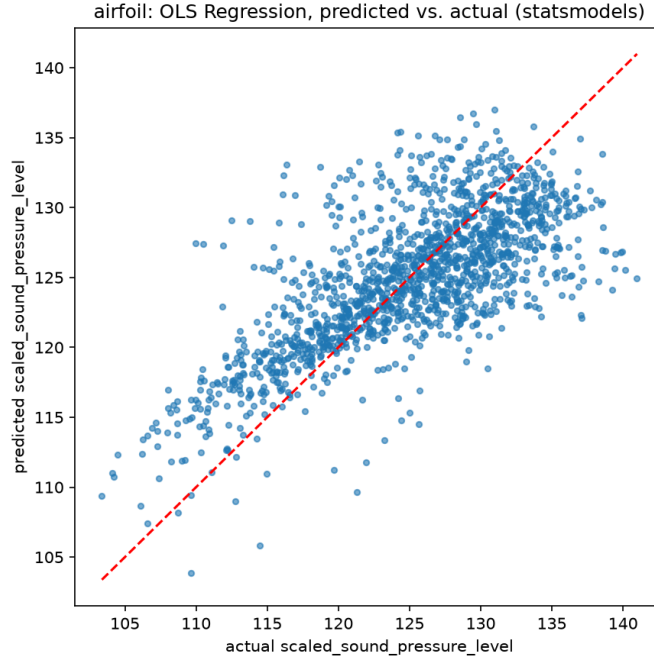


Figure 3: Airfoil: Statsmodels OLS, predicted vs. actual sound pressure level.

Interpretation: higher free-stream velocity is the only predictor that *raises* noise, as expected, since faster flow makes more self-noise. The other four all lower the fitted sound pressure level. Using all 5 predictors more than triples Project 1's best single-feature fit ($R^2 \approx 0.15$ for frequency alone), but this is still the weakest linear fit of the three datasets. That agrees with the Project 1 finding that boundary-layer self-noise is nonlinear.

2 Regularized Regression (Auto MPG only)

Ridge (L2) and Lasso (L1) were fit on Auto MPG in both tools. In ScalaTion, `RidgeRegression` and `LassoRegression` auto-center x and y (coefficients are on the original predictor scale, no intercept term); Statsmodels' `OLS.fit_regularized` was run on standardized predictors with y centered (it has no intercept column, so in each cross-validation fold y is centered on the training-fold mean and that mean is added back to the predictions). For each of the four models, the penalty (λ for ScalaTion, α for Statsmodels) was chosen by 5-fold cross-validated SSE/MSE over a grid (see `results/auto_mpg/{ridge,lasso}_alpha_search.csv` and the `scalation_output.txt` lambda-search log).

Table 4: Auto MPG Ridge and Lasso: chosen penalty, cross-validated error, and in-sample fit. ScalaTion's CV error is the mean SSE over 5 test folds of 78 rows each (about 11.2 as an MSE); Statsmodels' is the 5-fold mean MSE.

Tool	Model	Penalty (CV-chosen)	CV error	R^2
ScalaTion	Ridge	$\lambda = 10.0$	SSE 876.9	0.8214
ScalaTion	Lasso	$\lambda = 5.0$	SSE 877.0	0.8215
Statsmodels	Ridge	$\alpha = 0.01$	MSE 11.49	0.8208
Statsmodels	Lasso	$\alpha = 0.1$	MSE 11.46	0.8178
(unregularized OLS, Section 1)		–	–	0.8215

Table 5: ScalaTion coefficients on the original predictor scale: OLS (Section 1) vs. Ridge ($\lambda = 10$) vs. Lasso ($\lambda = 5$), Auto MPG. ScalaTion fits Ridge and Lasso on centered data, so they have no intercept term.

Predictor	OLS	Ridge	Lasso
cylinders	−0.49338	−0.44133	−0.48573
displacement	0.019896	0.018433	0.019314
horsepower	−0.016951	−0.015819	−0.019394
weight	−0.0064740	−0.0064996	−0.0064628
acceleration	0.080576	0.080558	0.030870
model_year	0.75077	0.74926	0.76718
origin	1.4261	1.3298	1.2816

Table 6: Statsmodels standardized coefficients: Ridge vs. Lasso (Auto MPG).

Predictor	Ridge	Lasso
cylinders	-0.684	0.000
displacement	1.357	0.000
horsepower	-0.754	-0.381
weight	-4.910	-4.718
acceleration	0.105	0.000
model_year	2.700	2.646
origin	1.112	0.892

Interpretation: with only 7 predictors on 392 rows, unregularized OLS is not overfitting, so shrinkage barely moves R^2 (0.818–0.822 across all four models) or cross-validated error (both Statsmodels models are within 0.04 MSE of each other). Cross-validation picks a very light Ridge penalty ($\alpha = 0.01$), which leaves the OLS coefficients nearly intact, including the counter-intuitive positive `displacement` coefficient from Section 1. Where regularization does help is in showing which predictors matter. Lasso’s L1 penalty drives `cylinders`, `displacement`, and `acceleration` exactly to zero, leaving a sparse 4-predictor model (`horsepower`, `weight`, `model_year`, `origin`) with almost the same fit. This agrees with forward selection in Section 3, whose first three picks are `weight`, `model_year` and `origin`. Those three drive fuel economy, and the engine-size predictors are largely redundant with each other once weight is in the model.

ScalaTion’s Ridge and Lasso do not standardize predictors, so their λ values and coefficients are on a different scale from Statsmodels’. At its CV-chosen λ , ScalaTion’s Lasso doesn’t set any coefficient exactly to zero, but it tells a similar story. `acceleration`, the least significant predictor in Section 1 ($p = 0.415$), shrinks the most ($0.0806 \rightarrow 0.0309$, down 62%). `weight` and `model_year`, the two most significant predictors, are almost unchanged. Ridge shrinks the `cylinders`, `displacement`, `horsepower` and `origin` coefficients by about 7–11% (`cylinders` the most). It leaves `weight`, `acceleration` and `model_year` within 0.4% of their OLS values.

3 Forward Feature Selection

ScalaTion’s `forwardSelAll` was run on Auto MPG and Concrete, which have 7 and 8 predictors. Airfoil was left out because with only 5 predictors the selection order says little.

Selection criterion. At each step ScalaTion adds the predictor that most lowers sMAPE-IC, an information criterion equal to sMAPE plus a size penalty $2(k+1)/m$ (k predictors, m rows). It is computed on ScalaTion’s training split (the first 80% of rows), so it is *not* a cross-validated measure. For each step we also report $\bar{R}^2$ (training split) and the 5-fold cross-validated R_{cv}^2 , because the three criteria do not always agree on the best subset size. k counts predictors and excludes the intercept.

3.1 Auto MPG

Table 7: Auto MPG forward selection path. Best value of each criterion in bold.

k	Predictor added	sMAPE-IC ↓	$\bar{R}^2$ (%) ↑	R_{cv}^2 (%) ↑
1	weight	11.918	75.30	68.49
2	model_year	10.931	81.34	80.27
3	origin	10.603	81.70	81.00
4	cylinders	10.548	81.70	80.96
5	acceleration	10.485	81.70	80.75
6	displacement	10.500	81.72	80.88
7	horsepower	10.586	81.79	80.92

ScalaTion’s own criterion (sMAPE-IC) is lowest at $k = 5$, so forward selection keeps 5 of the 7 predictors and drops `displacement` and `horsepower`. $\bar{R}^2$ keeps creeping up to the full 7-predictor model, while cross-validated R_{cv}^2 peaks at only 3 predictors (`weight`, `model_year`, `origin`).

3.2 Concrete

Table 8: Concrete forward selection path. Best value of each criterion in bold.

k	Predictor added	sMAPE-IC ↓	$\bar{R}^2$ (%) ↑	R_{cv}^2 (%) ↑
1	superplasticizer	38.616	21.20	12.46
2	age	32.733	39.50	29.07
3	cement	29.722	51.18	47.11
4	blast_furnace_slag	28.447	55.80	54.16
5	fly_ash	27.056	59.15	56.40
6	water	26.739	61.45	60.22
7	fine_aggregate	26.772	61.51	60.18
8	coarse_aggregate	26.776	61.54	60.24

sMAPE-IC is lowest at $k = 6$, so forward selection keeps 6 of the 8 predictors and drops the two aggregate fractions. $\bar{R}^2$ and R_{cv}^2 are both highest for the full model, but only by 0.02–0.09 percentage points.

Interpretation: forward selection does shrink both models under ScalaTion’s criterion: Auto MPG from 7 to 5 predictors, and Concrete from 8 to 6. The predictors it drops add almost nothing. After Auto MPG’s third step and Concrete’s sixth, $\bar{R}^2$ and R_{cv}^2 move by less than 0.3 percentage points, so the choice between the reduced and full models is a near-tie that depends on the criterion. The *order* in which predictors enter tells us more, because it ranks them by how much each one adds. For Auto MPG, `weight`, `model_year` and `origin` come first, and cross-validation says those three alone predict as well as any larger subset. They are also three of the four predictors Lasso keeps in Section 2. The engine-size predictors (`displacement`, `horsepower`) enter last because they carry little information that `weight` doesn’t already. For Concrete, superplasticizer and age are selected before cement: once workability (superplasticizer) and curing time (age) are accounted for, cement adds less marginal predictive power than its Section 1 coefficient alone would suggest. The two aggregates are the ones dropped. They also had the smallest Section 1 coefficients and were the only non-significant predictors there.

4 Transformed Regression

ScalaTion’s `TranRegression` was run on Auto MPG and Airfoil. In every model the transformed variable is the response y (`mpg` and `scaled_sound_pressure_level`); the predictors are left untransformed. Three transforms were applied, each with ScalaTion’s default parameters:

- **log:** `LogForm()`, which computes $\log(y + 1)$. Its default shift is 1, so this is $\log(1 + y)$ rather than $\log y$. Since `mpg` ranges from 9 to 46.6 and sound level from 103 to 141 dB, the shift barely changes the shape.
- **sqrt:** `RootForm()`, which computes $\sqrt{y}$.
- **Box-Cox:** `BoxcoxForm()`, which computes $(y^\lambda - 1)/\lambda$ with $\lambda = 0.4$. The rubric allows either Box-Cox or Yeo-Johnson. We used Box-Cox, since Yeo-Johnson is not exposed as a ready-made `Transform` in this ScalaTion build.

Both targets are strictly positive, so all three transforms are well-defined. The model is fit on the transformed scale, and predictions are back-transformed to the original units before computing R^2 , so every R^2 below is comparable to Section 1.

Table 9: Effect of transforming the response on in-sample R^2 (original units).

Dataset	Untransformed	$\log(y + 1)$	$\sqrt{y}$	Box-Cox, $\lambda = 0.4$
Auto MPG (mpg)	0.8215	0.8552	0.8437	0.8468
Airfoil (dB)	0.5157	0.5082	0.5122	0.5114

Table 10: Auto MPG transformed-regression coefficients (on the transformed scale of y).

Term	$\log(y + 1)$	$\sqrt{y}$	Box-Cox, $\lambda = 0.4$
intercept	1.8399	1.0753	0.94870
cylinders	-0.026517	-0.059416	-0.089092
displacement	0.00061195	0.0017523	0.0024836
horsepower	-0.0013590	-0.0025118	-0.0039338
weight	-0.00024473	-0.00063668	-0.00092184
acceleration	-0.0010393	0.0027383	0.0023211
model_year	0.028368	0.073812	0.10687
origin	0.039712	0.12166	0.17049

Table 11: Airfoil transformed-regression coefficients (on the transformed scale of y).

Term	$\log(y + 1)$	$\sqrt{y}$	Box-Cox, $\lambda = 0.4$
intercept	4.8983	11.530	15.178
frequency	-0.000010457	-0.000058117	-0.000071918
angle_of_attack	-0.0033419	-0.018848	-0.023255
chord_length	-0.28547	-1.6019	-1.9784
free_stream_velocity	0.00080062	0.0044871	0.0055432
suction_side_displacement_thickness	-1.2338	-6.7665	-8.3960

Reading the coefficients: every transform keeps the same sign as the untransformed model for each predictor, with one exception: **acceleration** in the Auto MPG log model, whose coefficient is tiny and was not significant in Section 1 ($p = 0.415$). The log model is the easiest to read, because a coefficient b multiplies $y + 1$ by roughly e^b . In Auto MPG, each model year raises mpg by about 2.9% ($e^{0.0284}$), and each extra 1000 lbs lowers it by about 22% ($e^{-0.245}$). These are percentage effects, which fit fuel economy better than the constant per-pound effect of the linear model. In Airfoil, each extra 1000 Hz lowers the sound level by about 1.0%, and each extra 0.1 m of chord by about 2.8%.

Interpretation: the transforms help Auto MPG and do not help Airfoil. mpg is skewed to the right and behaves like a reciprocal, since it measures distance per unit of fuel, while fuel per unit of distance is the quantity that adds up physically. Every transform toward a more symmetric scale improves the fit, and log is best, gaining $\Delta R^2 \approx 0.034$ over the untransformed model. Regressing $\log(\text{mpg})$ or gallons per 100 miles instead of mpg is standard practice for the same reason. **scaled_sound_pressure_level** is *already* measured in decibels, a logarithmic scale, so a further log, sqrt or Box-Cox transform slightly worsens R^2 , RMSE and sMAPE on the original scale.

5 Symbolic Regression

Both a power/cross-term expansion in ScalaTion and a genetic symbolic search in PySR were run on all three datasets.

5.1 Types of Symbolic Regression

Symbolic regression covers several different search strategies; the main families are listed in Table 12.

Table 12: Comparison of equation search and generation strategies in AI4Science.

Search Strategy	Mechanism	Discovery Speed	Complexity Limit	Best For
Pathwise Matrix (FFX)	Sparse deterministic linear regression over fixed libraries	Instantaneous (seconds)	Low to Medium	Ultra-fast baselines, large datasets
Time-Delay Search (NARX-SR)	Recurrent, lag-aware tree or orthogonal regressions	Fast to Medium (minutes)	Medium to High	Time series with feedback loops and historical memory
Physics Decomposition (AI Feynman)	Structural breakdown via physics priors (symmetries, units) + NN fit	Slow (hours)	High	Uncovering exact analytic laws of physics from raw data
Evolutionary Tree (PySR)	Genetic mutations and crossover of operator trees	Slow to Medium (mins/hrs)	High	Open-ended scientific discovery
Sparse Dynamics (SINDy)	Sequentially thresholded ridge regression	Fast	Medium	Physical systems (ODEs/PDEs)
Neural Generation (DSR)	Token-by-token sequence generation via RL	Medium	Medium	Finding specific structural patterns
Agentic Loop (SR-Scientist)	LLM code writing, execution, and optimization	Slow (agent loops)	Very High	Noisy data needing domain knowledge

The two tools used in this project come from different rows of this table. ScalaTion’s **SymbolicRegression** belongs to the first family: it builds a fixed library of candidate terms ($x^{0.5}$, x^2 , $x_i x_j$) and fits them with deterministic least squares. It differs from FFX in how it prunes the library: it uses greedy forward selection rather than a regularization path. PySR is the evolutionary-tree approach. It mutates and recombines expression trees, so it can produce forms the fixed library cannot represent, such as quotients and logarithms.

Our runs show the speed and flexibility trade-off from the table. Each dataset’s full ScalaTion run, including cross-validation and forward selection, took under 20 seconds and gave the same answer every time, but it can only reweight the terms it was given. PySR needed one to two minutes per dataset, plus a one-time Julia compilation, and returned compact closed-form equations (Section 5.3) built from operators outside the library. None of our three datasets is a time series or an ODE system, so the time-delay and sparse-dynamics families don’t apply here.

5.2 ScalaTion

Each dataset’s predictors were expanded with $x^{0.5}$, x^2 , an intercept, and all pairwise cross terms $x_i x_j$, then `forwardSelAll` was used to rank terms. As in Section 3, ScalaTion adds terms by sMAPE-IC on the training split, and we also track 5-fold cross-validated R_{cv}^2 . Term counts below exclude the intercept. An earlier attempt that also included x^{-1} and $x^{-0.5}$ terms produced a severely multicollinear, numerically unstable design matrix (coefficients up to 10^{15} , NaN standard errors, and an outright crash on Concrete) since a variable’s reciprocal and direct powers are highly correlated; the power set was narrowed to $\{0.5, 2\}$ to keep the expansion well-conditioned.

Table 13: ScalaTion symbolic regression. “Full” is the model with every candidate term (R^2 on all rows). The subsets are the forward-selection steps preferred by sMAPE-IC and by R_{cv}^2 . The last row gives Section 1’s linear regression for comparison.

	Auto MPG	Concrete	Airfoil
Candidate terms	35	44	20
R^2 , full model (all rows)	0.895	0.886	0.641
Best k by sMAPE-IC	27	37	16
Best k by R_{cv}^2	11	43	16
R_{cv}^2 at sMAPE-IC’s k	0.859	0.863	0.629
R_{cv}^2 at R_{cv}^2 ’s k	0.869	0.867	0.629
R_{cv}^2 , full model	0.860	0.866	0.628
R_{cv}^2 , linear regression (Section 1)	0.809	0.602	0.511

Interpretation: the power/cross-term expansion improves on linear regression for all three datasets, and the gain holds up out of sample: cross-validated R^2 rises from 0.81 to 0.86 (Auto MPG), 0.60 to 0.87 (Concrete), and 0.51 to 0.63 (Airfoil). Concrete gains the most, consistent with its strength depending on interactions between ingredients and age.

Forward selection behaves differently on each dataset. For Auto MPG, ScalaTion’s criterion keeps 27 of 35 terms, but cross-validation peaks much earlier: the first 11 terms ($R_{cv}^2 = 0.869$) predict slightly better than either the 27-term subset or the full model (0.860), so the later terms mostly fit noise. For Concrete, sMAPE-IC stops at 37 of 44 terms, leaving out the last 7 added (superplasticizer_fly_ash, fly_ash.cement, coarse_aggregate^{0.5}, superplasticizer_cement, fine_aggregate², age_coarse_aggregate, age_cement). Cross-validation does not support dropping them: R_{cv}^2 is 0.863 at 37 terms and 0.866 for the full model. The two criteria disagree, and those 7 terms neither clearly help nor clearly hurt. For Airfoil, all three criteria agree on the same 16 of 20 terms ($R_{cv}^2 = 0.629$, essentially equal to the full model’s 0.628). Airfoil is still the weakest of the three datasets, which suggests nonlinear structure that a low-order power/cross-term basis can’t capture.

5.3 PySR

PySR (binary operators $+$, $-$, $\times$, $\div$; unary operators square, sqrt, exp, log; 60 iterations, population size 33) searched directly over closed-form expressions rather than a fixed basis.

Table 14: PySR best equation per dataset (selected by PySR’s complexity/loss score).

Dataset	Equation	Complexity	R^2
Auto MPG	$\text{mpg} = \frac{(\text{model_year} - 45.714) \times 2104.74}{\text{weight}}$	7	0.8508
Concrete	$\text{strength} = \log(\text{fly_ash} + \text{water}) (\log(\text{age}) \text{ cement} + 958.16) / \text{water} + \sqrt{\text{blast_furnace_slag} - 24.85}$	18	0.7564
Airfoil	$\text{spl} = 131.38 - \sqrt{25.816 \text{ freq} \cdot \text{chord} \cdot \text{thickness}}$	10	0.5556

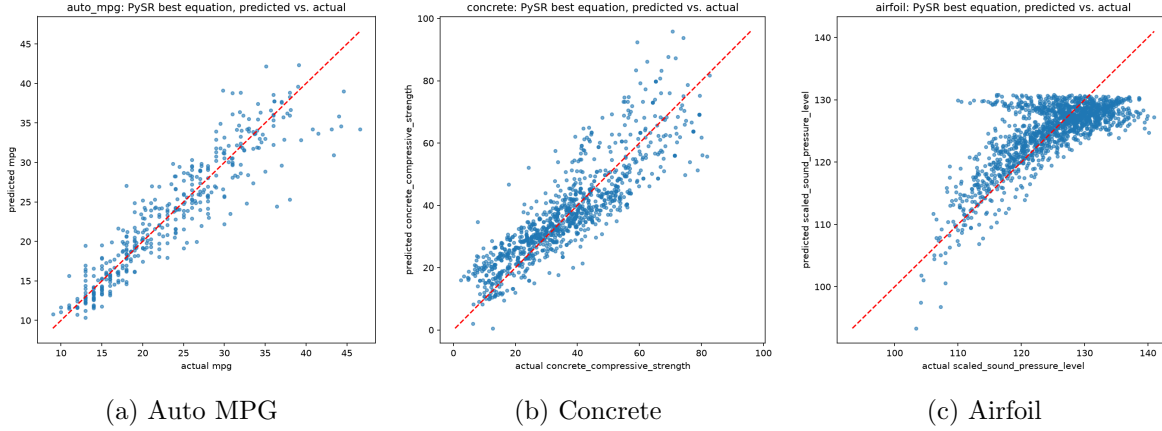


Figure 4: PySR best-equation predictions vs. actual, all three datasets.

Interpretation: all PySR R^2 values are in-sample on all rows, so they compare directly with the full-model R^2 in Table 13 and with Section 1.

PySR’s Auto MPG equation is close to physical reasoning: it finds that fuel economy scales with a (model-year-adjusted) constant divided by weight. It uses only 2 fitted constants, yet reaches $R^2 = 0.851$, *beating* the 8-parameter linear model ($R^2 = 0.822$), though not ScalaTion’s 35-term expansion ($R^2 = 0.895$). For Concrete, PySR clearly beats the linear model ($R^2 = 0.756$ vs. 0.616) but falls well short of ScalaTion’s 44-term expansion ($R^2 = 0.886$). Its equation is much shorter, though, and contains a $\log(\text{age}) \times \text{cement}/\text{water}$ interaction that ScalaTion’s $\{x^{0.5}, x^2, x_i x_j\}$ basis cannot express. That matches concrete science: strength gain follows a roughly logarithmic curing curve in age and depends on the cement/water ratio. For Airfoil, PySR’s equation ($R^2 = 0.556$) improves modestly on linear regression (0.516) but is below ScalaTion’s 20-term expansion ($R^2 = 0.641$). The **frequency** $\times$ **chord.length** $\times$ **thickness** product it finds is consistent with airfoil self-noise scaling, in which noise depends on these geometric and aerodynamic quantities jointly.

Cross-Section Summary

- **Accuracy by dataset:** linear regression fits Auto MPG best, then Concrete, then Airfoil. Once interaction terms are allowed (Section 5), Concrete catches up with Auto MPG (cross-validated R^2 0.866 vs. 0.860), since much of its structure is in ingredient and age interactions. Airfoil stays last under every method.
- **Regularization** (Section 2) barely changed accuracy, since the unregularized Auto MPG model was not overfitting. It was more useful for interpretation: Lasso’s sparsity shows which of the correlated engine-size features are redundant.
- **Forward selection** (Section 3) shrank Auto MPG from 7 to 5 predictors and Concrete from 8 to 6 under ScalaTion’s sMAPE-IC criterion. The dropped predictors change $\bar{R}^2$ and R_{cv}^2 by under 0.3 percentage points. Its selection order agrees with Lasso: weight, model year and origin carry most of Auto MPG’s signal.
- **Transformed regression** (Section 4) helped Auto MPG, whose target is skewed, and did not help Airfoil, whose target is already on a logarithmic decibel scale.
- **Symbolic regression** (Section 5) gave the largest accuracy gains of any technique, and the gains hold up under cross-validation. ScalaTion’s power/cross-term expansion was the most accurate, and PySR’s equations were much more compact. Only PySR found terms outside a fixed polynomial basis: a 1/weight scaling for Auto MPG and a $\log(\text{age})$ curing term for Concrete, both of which match the physics of the problem.